

Machine Learning ETE Question 4

Why this combination?

- PCA is an appropriate unsupervised dimensionality-reduction method for continuous, correlated numeric variables. Standardisation is required because PCA is scale-sensitive.
- Random Forest Regressor is an ensemble of decision trees that models non-linear relationships and feature interactions with little tuning.
- PCA is fitted inside a Pipeline, so it learns only from the training split and cannot leak test-set information. The baseline shows whether reduction is worthwhile for this small, eight-feature dataset.

```
In [16]: import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

RANDOM_STATE = 42
```

```
In [17]: housing = fetch_california_housing(as_frame=True)
X, y = housing.data, housing.target

print(f'Samples: {X.shape[0]:,}')
print(f'Original features: {X.shape[1]}')
display(X.head())
```

```
Samples: 20,640
Original features: 8
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitu
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122

Exploratory Data Analysis

```
In [18]: eda_summary = pd.DataFrame({
    'Data type': X.dtypes.astype(str),
    'Missing values': X.isna().sum(),
    'Unique values': X.nunique(),
})
display(eda_summary)
display(X.describe().T)
print(f'Target mean: ${y.mean() * 100_000:,.0f}')
print(f'Target range: ${y.min() * 100_000:,.0f} to ${y.max() * 100_000:,.0f}')
```

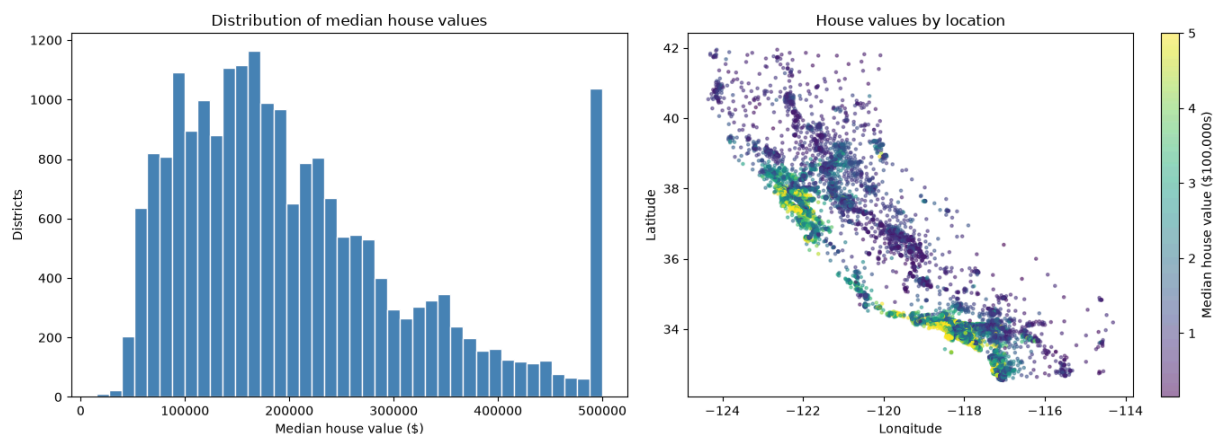
	Data type	Missing values	Unique values
MedInc	float64	0	12928
HouseAge	float64	0	52
AveRooms	float64	0	19392
AveBedrms	float64	0	14233
Population	float64	0	3888
AveOccup	float64	0	18841
Latitude	float64	0	862
Longitude	float64	0	844

	count	mean	std	min	25%	50%	
MedInc	20640.0	3.870671	1.899822	0.499900	2.563400	3.534800	
HouseAge	20640.0	28.639486	12.585558	1.000000	18.000000	29.000000	
AveRooms	20640.0	5.429000	2.474173	0.846154	4.440716	5.229129	
AveBedrms	20640.0	1.096675	0.473911	0.333333	1.006079	1.048780	
Population	20640.0	1425.476744	1132.462122	3.000000	787.000000	1166.000000	1
AveOccup	20640.0	3.070655	10.386050	0.692308	2.429741	2.818116	
Latitude	20640.0	35.631861	2.135952	32.540000	33.930000	34.260000	
Longitude	20640.0	-119.569704	2.003532	-124.350000	-121.800000	-118.490000	-

Target mean: \$206,856
Target range: \$14,999 to \$500,001

```
In [19]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].hist(y * 100_000, bins=40, color='steelblue', edgecolor='white')
axes[0].set(title='Distribution of median house values', xlabel='Median house value

price_map = axes[1].scatter(X['Longitude'], X['Latitude'], c=y, cmap='viridis', s=5
axes[1].set(title='House values by location', xlabel='Longitude', ylabel='Latitude'
fig.colorbar(price_map, ax=axes[1], label='Median house value ($100,000s)')
plt.tight_layout()
plt.show()
```



Preprocessing Checks

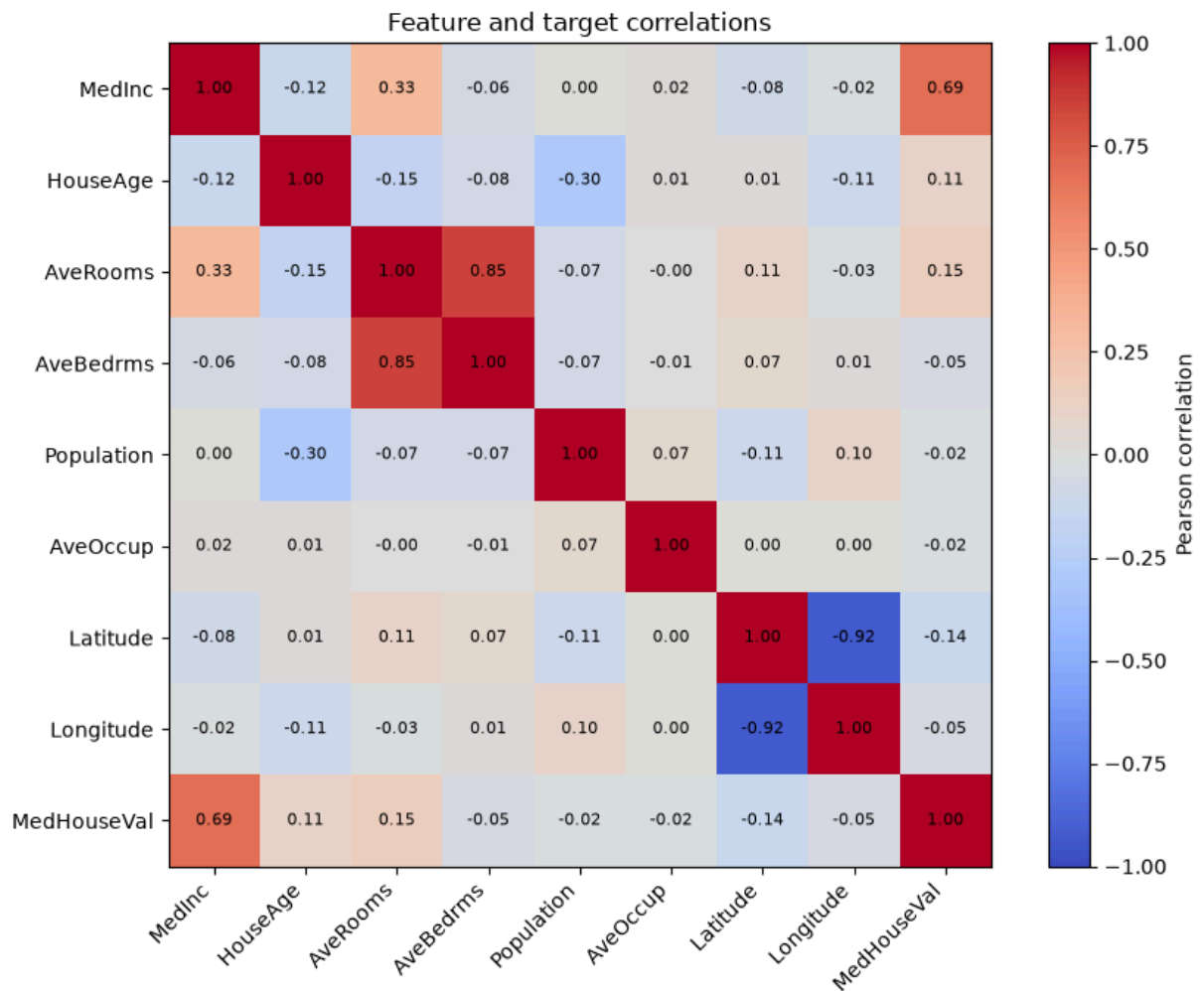
```
In [20]: preprocessing_checks = pd.Series({
    'Missing feature values': int(X.isna().sum().sum()),
    'Duplicate feature rows': int(X.duplicated().sum()),
    'Non-numeric features': X.select_dtypes(exclude=np.number).shape[1],
    'Largest/smallest feature standard-deviation ratio': X.std().max() / X.std().mi
})
```

```
display(preprocessing_checks.to_frame('Result').round(2))
print('Scaling before PCA is required:', preprocessing_checks.iloc[-1] > 10)
```

	Result
Missing feature values	0.00
Duplicate feature rows	0.00
Non-numeric features	0.00
Largest/smallest feature standard-deviation ratio	2389.61

Scaling before PCA is required: True

```
In [21]: correlation = X.assign(MedHouseVal=y).corr(numeric_only=True)
fig, ax = plt.subplots(figsize=(9, 7))
heatmap = ax.imshow(correlation, cmap='coolwarm', vmin=-1, vmax=1)
ax.set_xticks(range(len(correlation)), correlation.columns, rotation=45, ha='right')
ax.set_yticks(range(len(correlation)), correlation.index)
for row, column in np.ndindex(correlation.shape):
    ax.text(column, row, f'{correlation.iloc[row, column]:.2f}', ha='center', va='c')
fig.colorbar(heatmap, ax=ax, label='Pearson correlation')
ax.set_title('Feature and target correlations')
plt.tight_layout()
plt.show()
```



```
In [22]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE
    )

    baseline = RandomForestRegressor(
        n_estimators=200, random_state=RANDOM_STATE, n_jobs=-1
    )
    pca_preprocessor = Pipeline([
        ('impute', SimpleImputer(strategy='median')),
        ('scale', StandardScaler()),
    ])
    pca_forest = Pipeline([
        ('preprocess', pca_preprocessor),
        ('pca', PCA(n_components=0.95, random_state=RANDOM_STATE)),
        ('forest', RandomForestRegressor(
            n_estimators=200, random_state=RANDOM_STATE, n_jobs=-1
        )),
    ])
```

```
In [23]: results = []
    for name, model, feature_count in [
        ('Random Forest (baseline)', baseline, X_train.shape[1]),
        ('PCA (95%) + Random Forest', pca_forest, None),
    ]:
        start = time.perf_counter()
```

```

model.fit(X_train, y_train)
fit_seconds = time.perf_counter() - start
predictions = model.predict(X_test)
if feature_count is None:
    feature_count = model.named_steps['pca'].n_components_
results.append({
    'Model': name,
    'Features used': feature_count,
    'Variance retained': 1.0 if name.startswith('Random') else model.named_step
    'R²': r2_score(y_test, predictions),
    'RMSE': mean_squared_error(y_test, predictions) ** 0.5,
    'Fit time (s)': fit_seconds,
})

comparison = pd.DataFrame(results).set_index('Model')
assert np.isfinite(comparison[['R²', 'RMSE', 'Fit time (s)']].to_numpy()).all()
display(comparison.round(4))

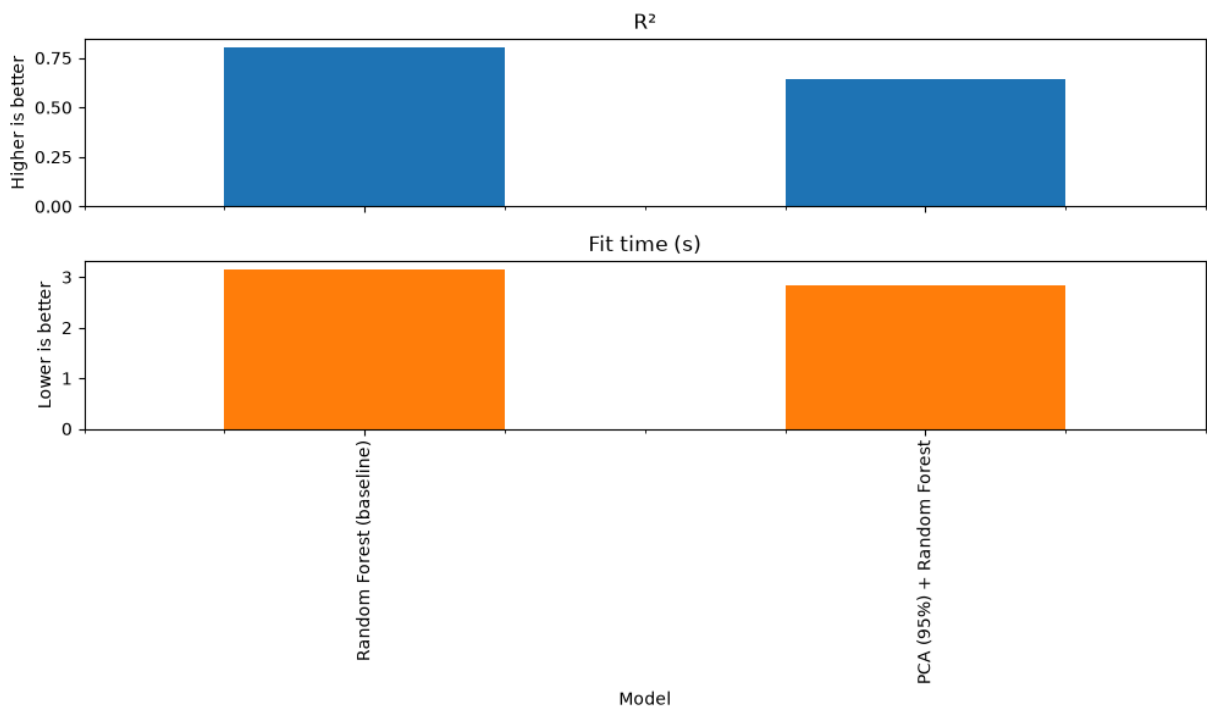
```

Model	Features used	Variance retained	R²	RMSE	Fit time (s)
Random Forest (baseline)	8	1.0000	0.8059	0.5043	3.1555
PCA (95%) + Random Forest	6	0.9837	0.6437	0.6833	2.8340

```

In [24]: ax = comparison[['R²', 'Fit time (s)']].plot.bar(subplots=True, legend=False, figsi
ax[0].set_ylabel('Higher is better')
ax[1].set_ylabel('Lower is better')
plt.tight_layout()
plt.show()

```



Discussion

- **Accuracy:** PCA may improve generalisation when noisy or highly collinear features dominate. It can also lower R^2 here because principal components preserve feature variance, not their relationship with house price. Use the displayed hold-out R^2 and RMSE to choose the better model.
- **Computation cost:** retaining 95% of variance reduces the number of inputs evaluated at each tree split, which can reduce memory and fitting cost for wide datasets. On this dataset PCA itself adds a small upfront cost and only eight features exist, so the baseline can be just as fast or faster.
- **Interpretability:** the baseline's feature importances refer to original variables such as `MedInc` and `AveRooms`. PCA features are weighted mixtures of those variables, so the PCA pipeline is less directly interpretable even when accuracy is similar.

Conclusion: keep PCA + Random Forest only if its validation performance is comparable or better and dimensionality reduction is valuable. For these eight original features, the baseline is usually easier to explain.